Volcano for the Agentic AI Era:

Unified Scheduling Across Training, Inference, and Agents

Ricardo Aravena · Rainie Li

KubeCon + CloudNativeCon North America 2026
Salt Lake City · November 9–12

Speakers

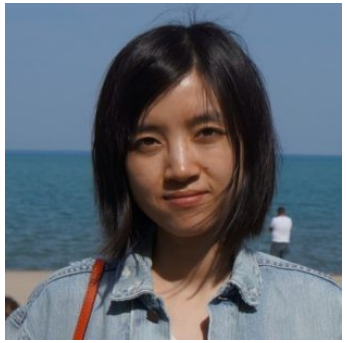


Ricardo Aravena

AI Infrastructure Lead · Nunchux AI

CNCF TOC Alternate (GB)
Cloud native and AI infrastructure

nunchux.ai · CNCF TOC



Rainie Li

Pinterest

Data processing infrastructure
Apache YuniKorn PMC member

Apache YuniKorn · rainieli

01

Unified scheduling

Different paths. Shared infrastructure.

01 Unified scheduling

02 Multi-level gang & network topology

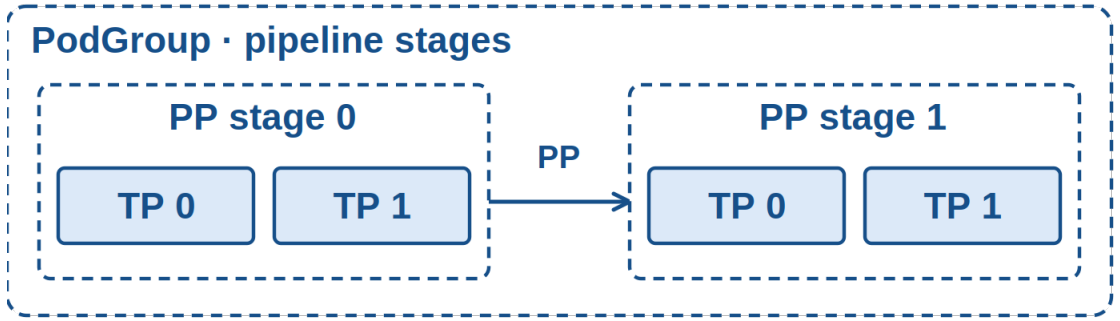
03 Ecosystem

04 Benchmark framework & progress

Different Workloads, Shared Infrastructure

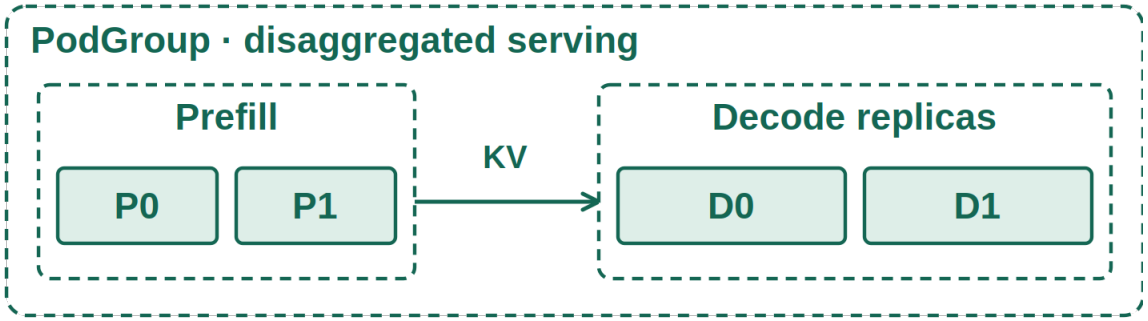
Gang size · network locality · startup latency

Training



TP / PP gangs · locality

Inference



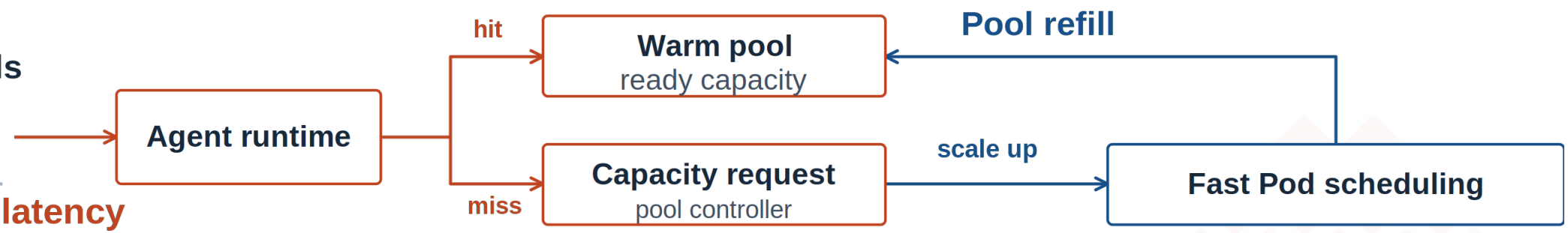
P/D groups · locality

Agents

Bursty arrivals

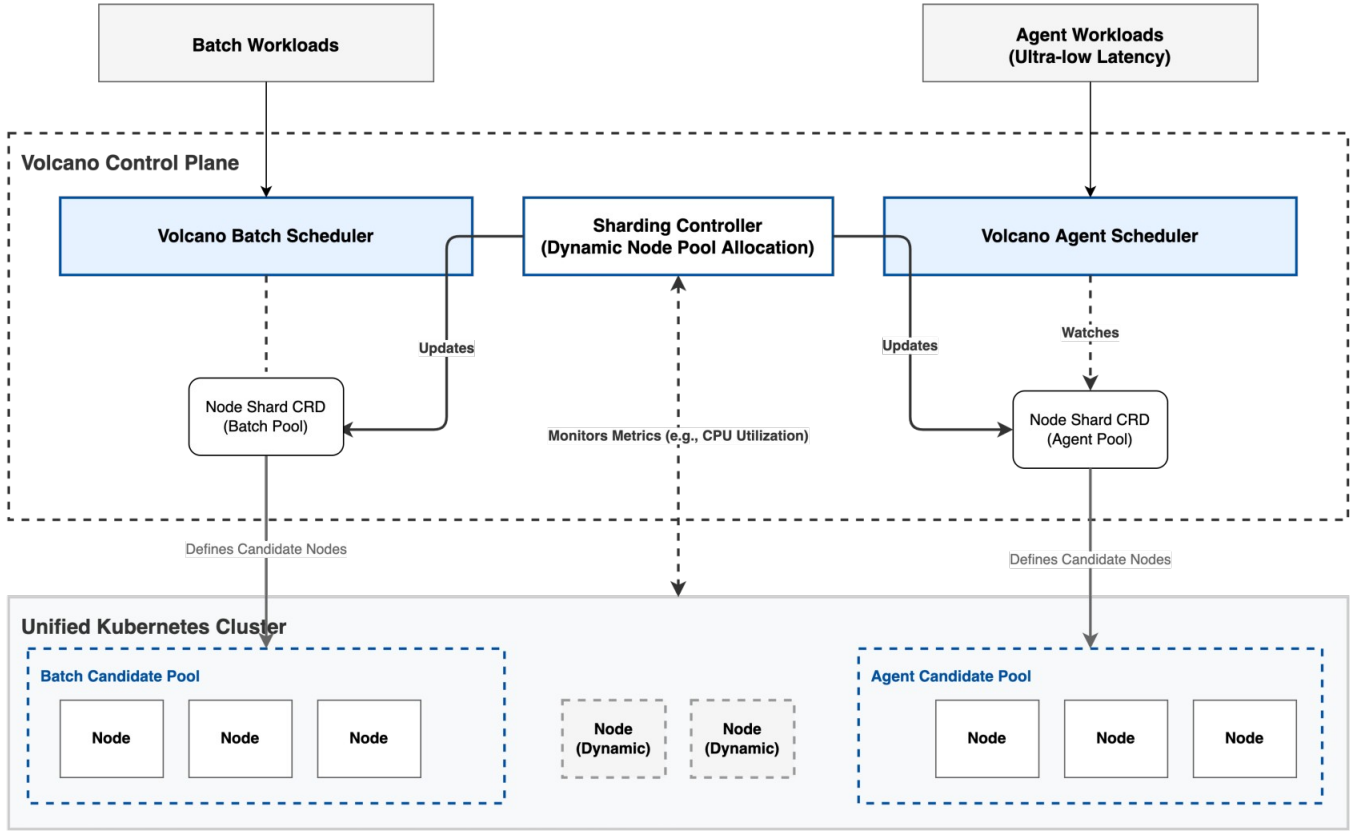


Low startup latency



A Coordinated Scheduling Framework

Batch path · fast path · shared NodeShard



Workload paths

Batch · fast path

Worker coordination

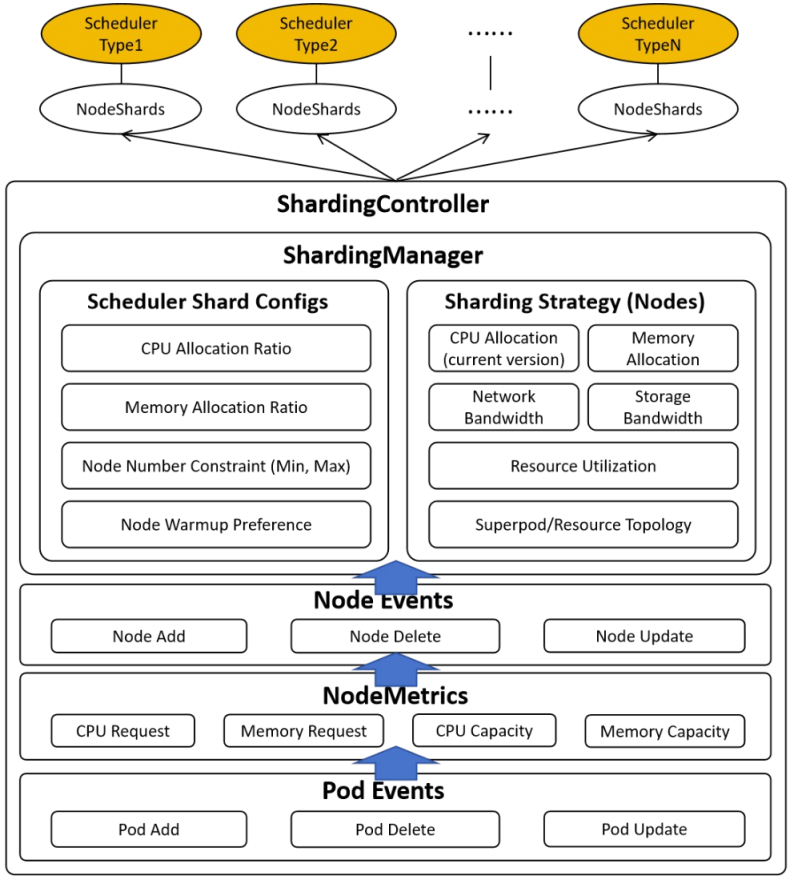
Binder conflict checks

Shared node access

NodeShard

NodeShard: Desired Nodes and Current Use

Desired membership → coordinated handoff



NodeShard · spec / status excerpt

```
spec:
  nodesDesired:
  - node1
  - node2
status:
  nodesInUse:
  - node0
  - node1
  nodesToRemove:
  - node0
  nodesToAdd:
  - node2
```

Desired ≠ in use

02

Multi-level gang & network topology

From available nodes to runnable workloads.

01 Unified scheduling

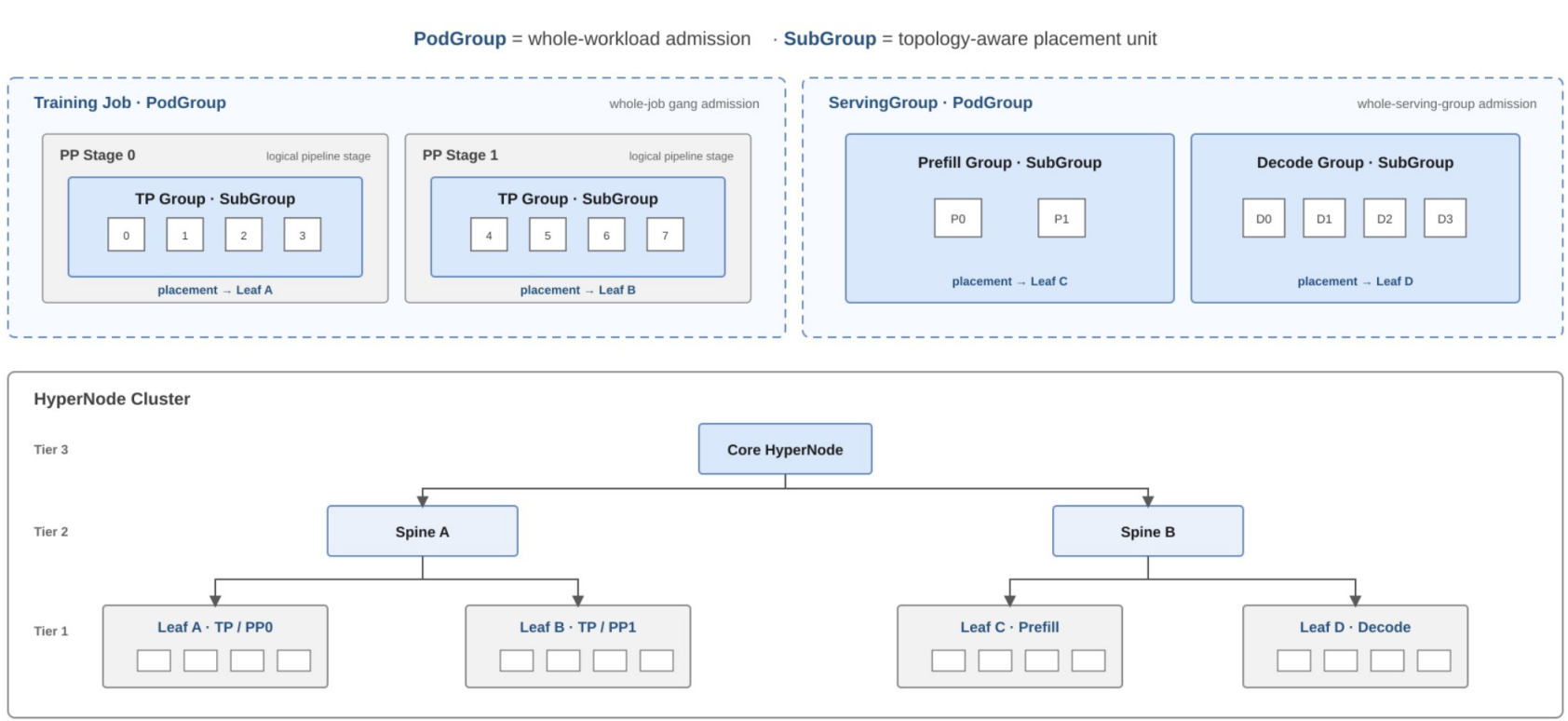
02 Multi-level gang & network topology

03 Ecosystem

04 Benchmark framework & progress

PodGroup and SubGroup: Preserve Workload Structure

Workload gang · replica gang

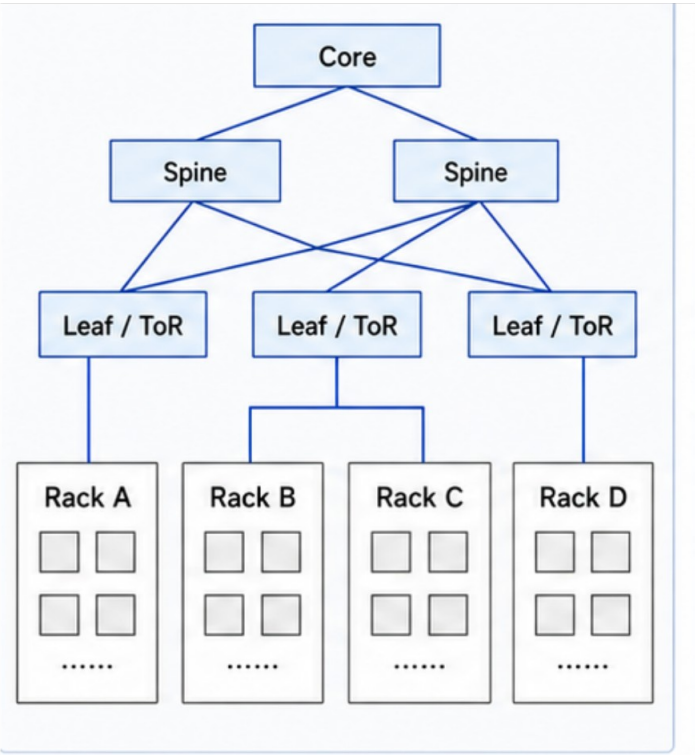


Pod selection + replica identity

HyperNode: From Physical Fabric to Scheduling Domains

Physical network → scheduling domains

Physical fabric



Standalone HyperNode

Parent · HyperNode.spec

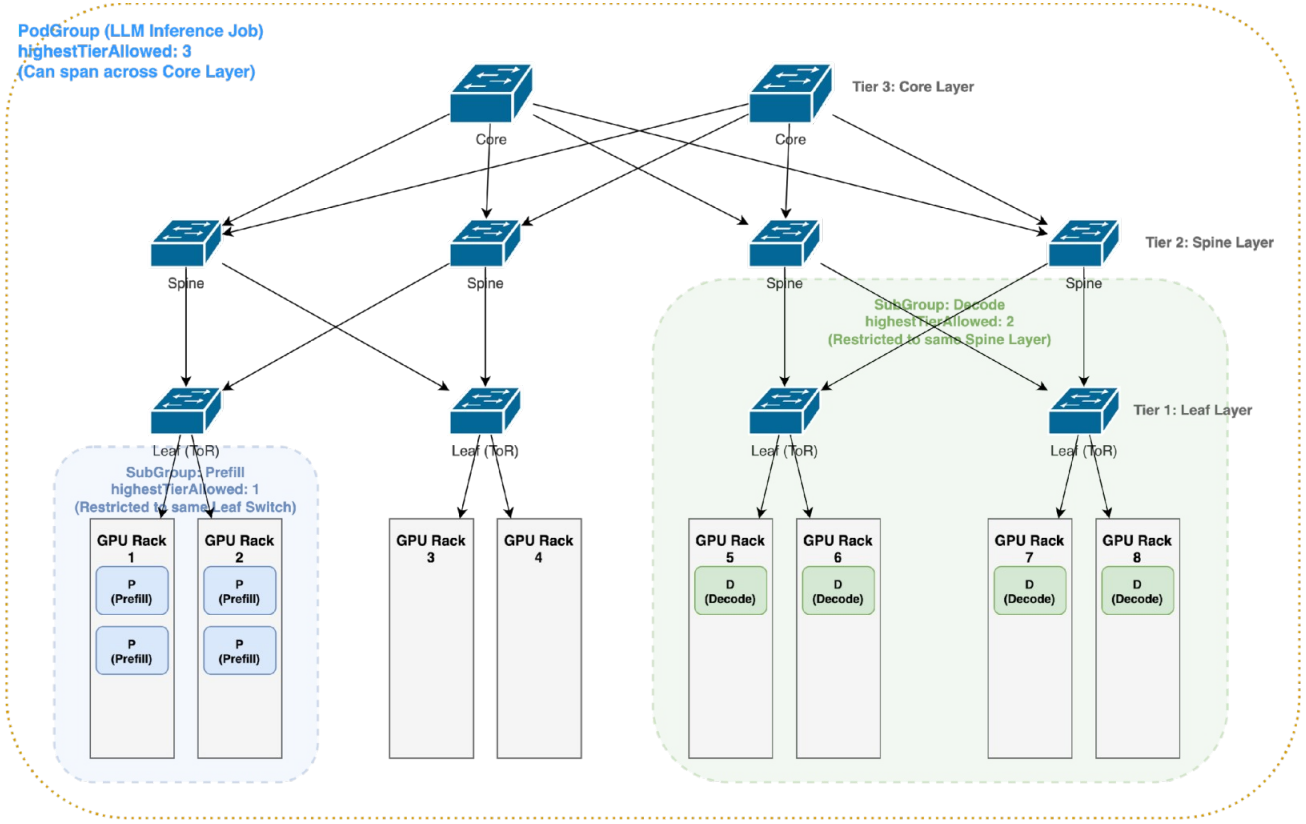
```
tier: 2
tierName: spine
members:
- type: HyperNode
  selector:
    exactMatch:
      name: rack-a
```

Leaf · HyperNode.spec

```
tier: 1
tierName: leaf
members:
- type: Node
  selector:
    labelMatch:
      matchLabels:
        rack: rack-a
```

Place Groups at Different Topology Levels

PodGroup: core · Prefill: leaf · Decode: spine



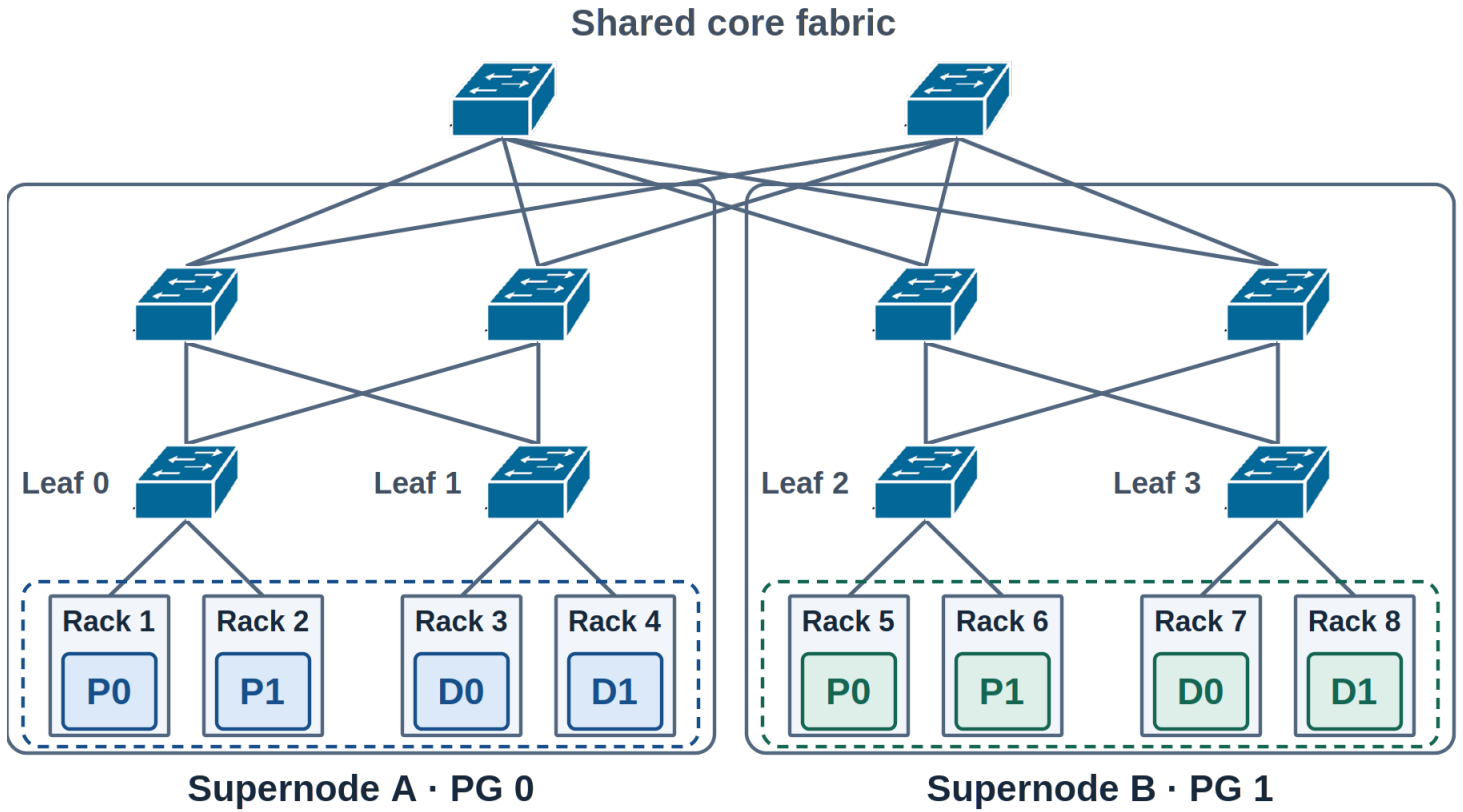
Core (3) / Spine (2) / Leaf (1)

PodGroup.spec · key fields

```
minMember: 8
networkTopology:
  highestTierName: core
subGroupPolicy:
- name: prefill
  subGroupSize: 4
  networkTopology:
    highestTierName: leaf
- name: decode
  subGroupSize: 4
  networkTopology:
    highestTierName: spine
```

Keep Each Instance Together, Place Instances Apart

PodGroup anti-affinity · Kthena integration proposed



PodGroup.spec · key fields

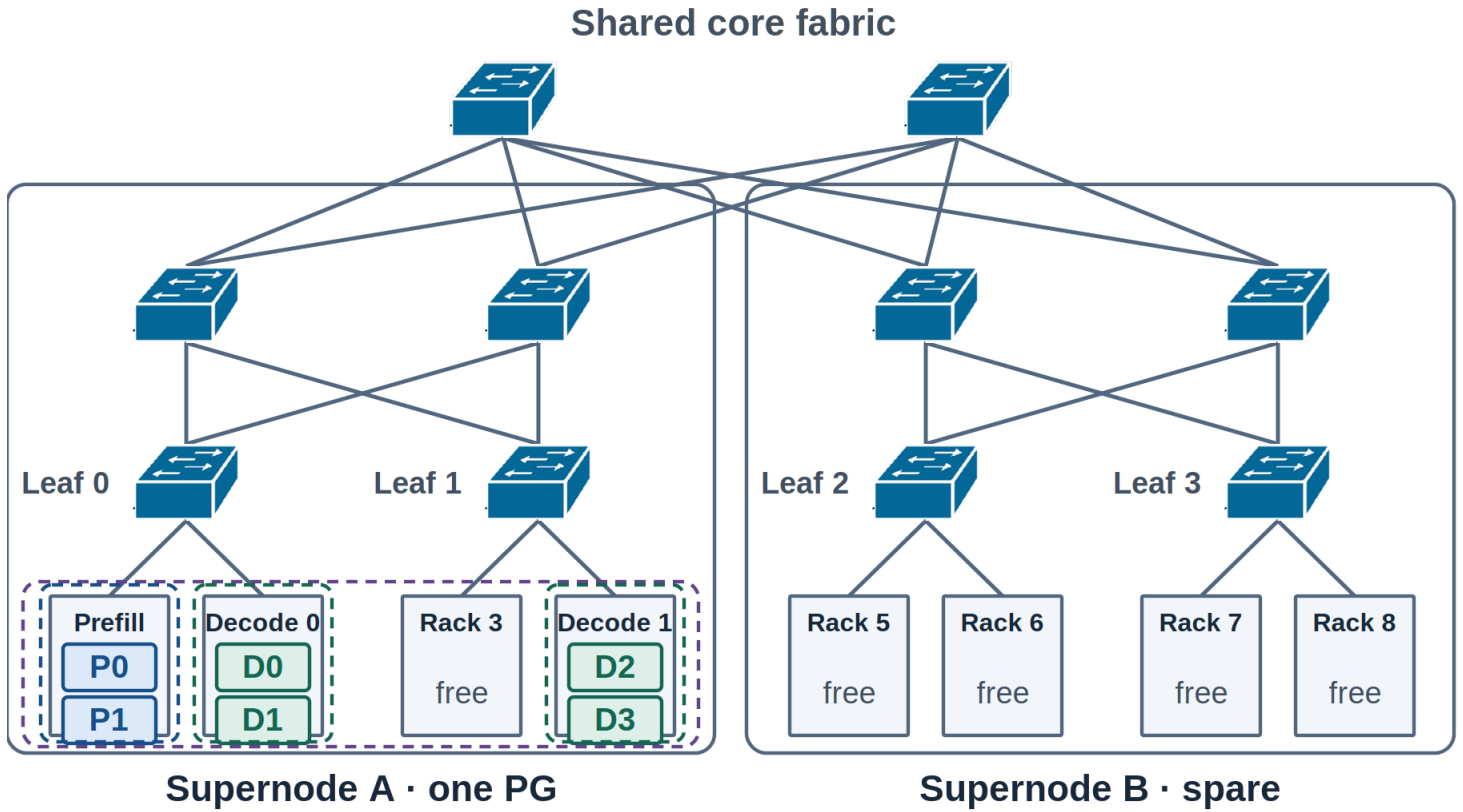
```
networkTopology:
  mode: hard
  highestTierName: supernode
topologyAffinity:
  podGroupAntiAffinity:
    required:
    - podGroupSelector:
        matchLabels:
          app: model-a
      topologyTierName: supernode
```

PG label: app=model-a

Separate failure domains

Next: Relationships Between SubGroups

SubGroup affinity / anti-affinity · proposed



Affinity

Prefill + Decode

Same supernode · KV locality

Anti-affinity

Decode replicas

Separate leaves

03

Ecosystem

Workload controllers carry scheduling intent.

01 Unified scheduling

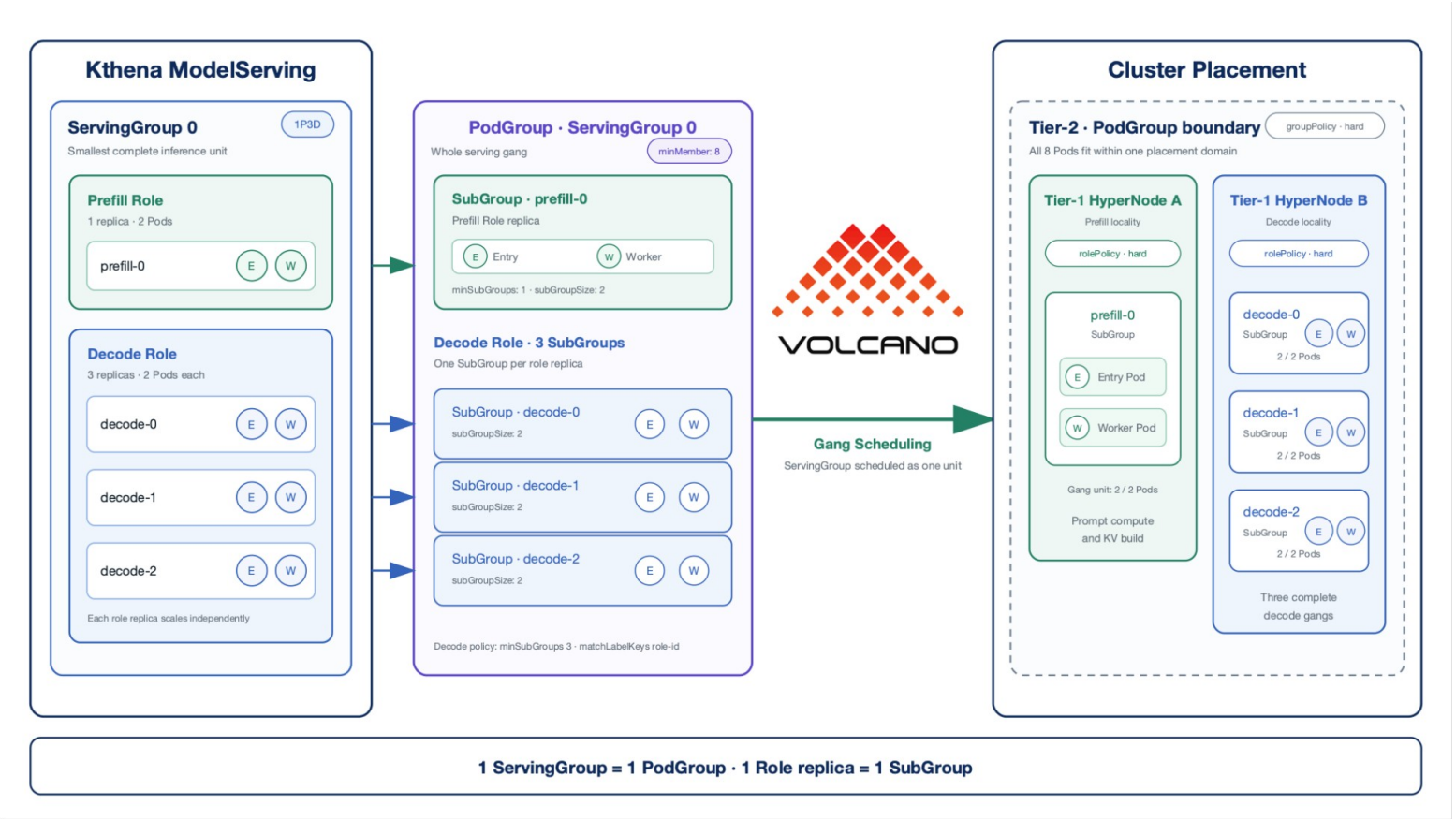
02 Multi-level gang & network topology

03 Ecosystem

04 Benchmark framework & progress

Kthena: From Serving Roles to Scheduling Groups

ServingGroup → PodGroup · Role replica → SubGroup



ModelServing.spec · excerpt

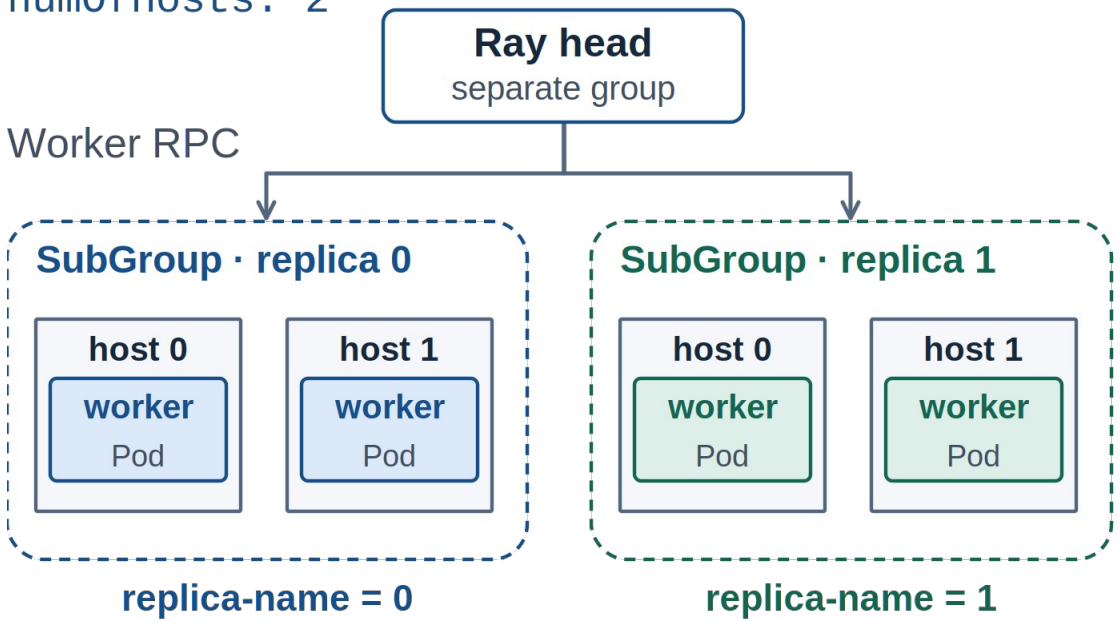
```
schedulerName: volcano
replicas: 1
template:
  gangPolicy:
    minRoleReplicas:
      prefill: 1
      decode: 3
  roles:
  - name: prefill
    replicas: 1
    workerReplicas: 1
  - name: decode
    replicas: 3
    workerReplicas: 1
```

KubeRay: A Replica Can Span Multiple Pods

Multi-host replicas · label-based grouping

RayCluster · worker group

```
minReplicas: 2
numOfHosts: 2
```



PodGroup.spec · generated grouping

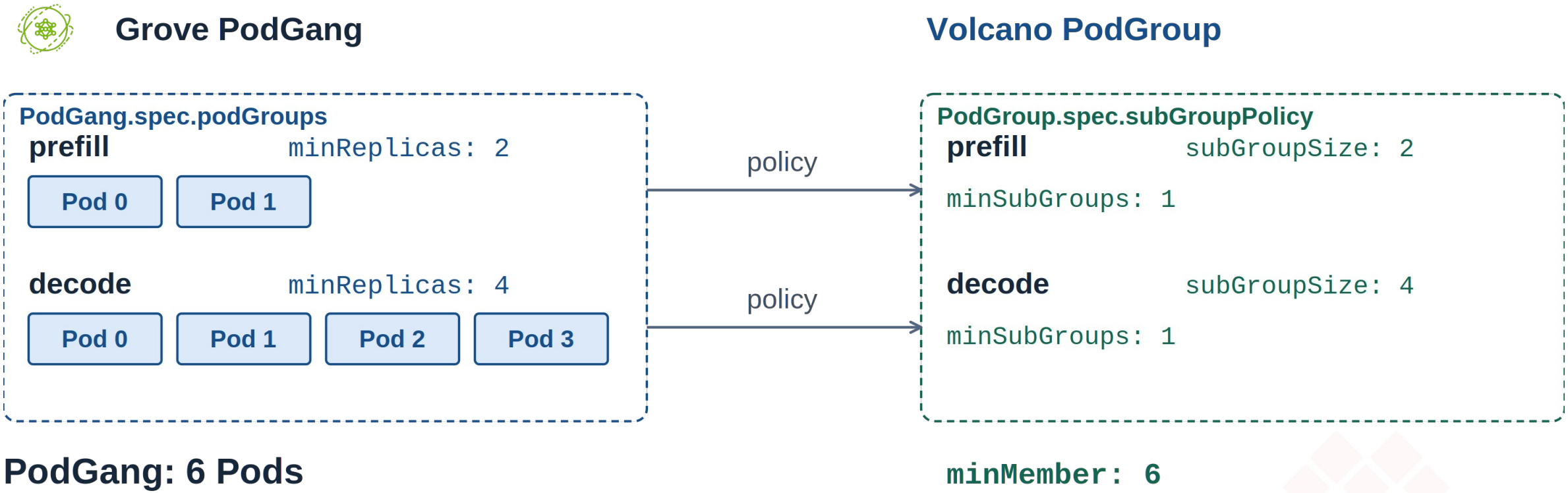
```
subGroupPolicy:
- name: worker
  labelSelector:
    matchLabels:
      ray.io/group: worker
  matchLabelKeys:
  - ray.io/worker-group-replica-name
  minSubGroups: 2
  subGroupSize: 2
```

Replica label → SubGroup

Gang: merged · Topology: in progress

Grove: Carry PodGang Structure into Volcano

Gang / SubGroup mapping available · topology unsupported



Next: RBG, Kubeflow, and LWS

RBG in progress · Kubeflow and LWS planned



RBG

In progress

RoleInstance → SubGroup

Gang integration



Kubeflow

Planned

Training groups + topology

Volcano plugin



LWS

Planned

Multi-role / multi-group

Cross-group placement

04

Benchmark framework & progress

Measure behavior across workload shapes.

01 Unified scheduling

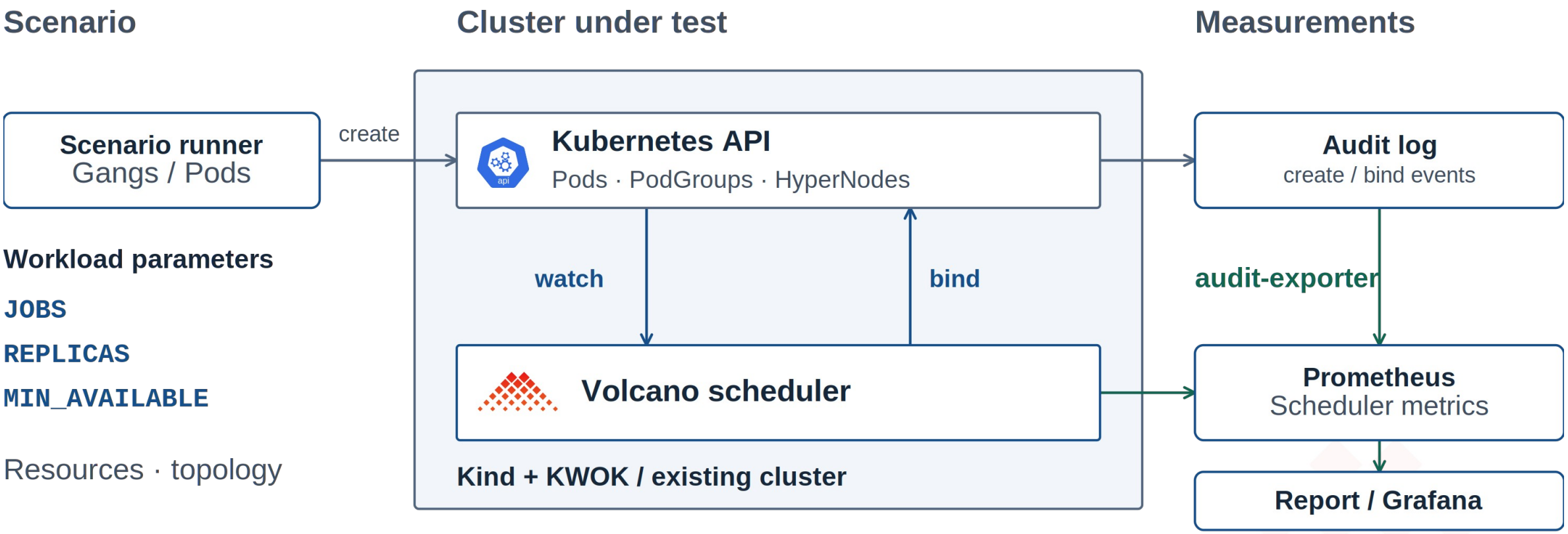
02 Multi-level gang & network topology

03 Ecosystem

04 Benchmark framework & progress

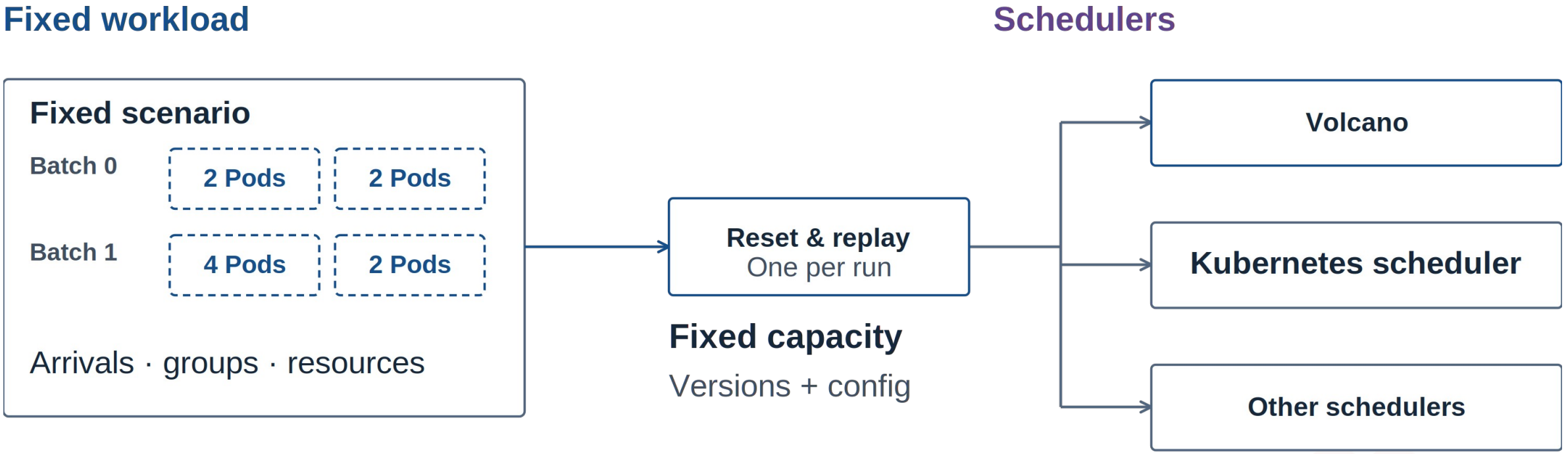
A Repeatable Scheduling Benchmark Framework

Scenario → cluster → measurements



Compare Schedulers on the Same Workload

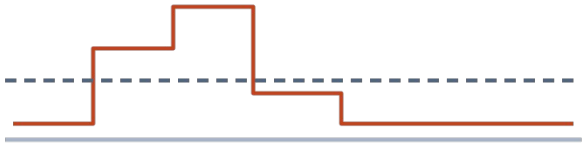
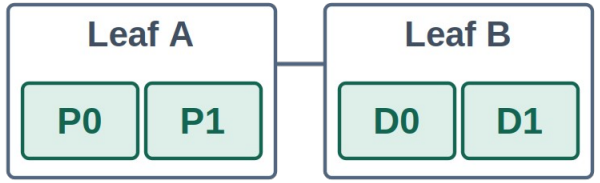
Cross-scheduler comparisons · planned



Latency · Throughput · Complete gangs · Placement

Which Scenarios Fit Which Scheduler?

Bursts · group size · topology

Scenario		Variables	Metrics
Bursts		**Arrival rate · capacity** Pool capacity requests	**Tail latency** Backlog recovery
Gang size		**Group size · fragmentation** Large / small gangs	**Complete gangs** Binding throughput
Topology		**Tier limits · domains** Gang · anti-affinity	**Valid placement** Gang latency

What We Will Work On Next

Group placement · ecosystem · benchmarks

Workstream	Next	Validation
Group placement	**SubGroup relationships** Affinity / anti-affinity	Locality Replica isolation
Ecosystem	**RBG · Kubeflow · LWS** Multi-group integration	Group identity Placement intent
Benchmarks	**Cross-scheduler testing** Best-fit scenarios · topology	Latency · throughput Placement correctness

Volcano community



Thank you



volcano.sh



GitHub



Slack

Questions?